Laboratoire PowerShell

Administration Windows en ligne de commande
Volume 1, modules 1 et 2: bases du langage et gestion des fichiers et dossiers

Cmdlets Verbe-Nom · Aide intégrée · Navigation · Fichiers et dossiers · Lecture et écriture

Alexandre Peralta
Technicien gestionnaire PC & Réseaux en formation - CEFAID
Juin 2026

Résumé du projet

Réalisation, test et documentation des deux premiers modules d'un parcours PowerShell, en ligne de commande sous Windows. Ce volume présente d'abord les bases du langage (logique Verbe-Nom, recherche de commandes, aide intégrée), puis la gestion des fichiers et dossiers: navigation, création, lecture, écriture, copie, renommage, déplacement et suppression. Chaque commande a été exécutée et vérifiée sur un dossier de test, créé puis supprimé, avec une capture d'écran à l'appui.

2	16	26	1
modules couverts	commandes documentées	captures d'écran documentées	dossier de test créé puis supprimé

Compétences clés

Logique Verbe-Nom	**Get-Command**	**Get-Help**
Navigation	**Fichiers et dossiers**	**Lecture et écriture**
Copie et déplacement	**Suppression**	**Documentation technique**

Pour aller plus loin dans ce document

- Sommaire complet: page suivante
- Qu'est-ce que PowerShell, objets et pipeline: sections 1.1 à 1.4
- Aide intégrée et téléchargement de l'aide: sections 3.3 et 3.4
- Navigation dans les dossiers: sections 4.1 à 4.4
- Création, écriture et lecture: sections 4.5 à 4.10
- Copie, renommage et déplacement: sections 4.11 à 4.16
- Suppression et paramètre -Recurse: sections 4.17 à 4.21
- Tableau récapitulatif des commandes: chapitre 5
- Difficultés rencontrées et solutions: chapitre 6
- Suite du cours (volumes 2 et 3): chapitre 7

Sommaire

1. Contexte et objectifs
- 1.1 Qu'est-ce que PowerShell?
- 1.2 À quoi sert PowerShell?
- 1.3 Un shell et un langage de script
- 1.4 La particularité de PowerShell: les objets

2. Environnement technique

3. Les bases de PowerShell
- 3.1 La logique Verbe-Nom: Get-Date
- 3.2 Retrouver une commande: Get-Command
- 3.3 L'aide intégrée: Get-Help
- 3.4 Télécharger l'aide: Update-Help
- 3.5 Effacer l'écran: cls

4. Fichiers et dossiers
- 4.1 Où suis-je? Get-Location
- 4.2 Lister le contenu: Get-ChildItem
- 4.3 Changer de dossier: Set-Location
- 4.4 Remonter d'un niveau: Set-Location ..
- 4.5 Créer un dossier: New-Item
- 4.6 Créer un fichier: New-Item -ItemType File
- 4.7 Écrire dans un fichier: Set-Content
- 4.8 Lire un fichier: Get-Content
- 4.9 Ajouter une ligne: Add-Content
- 4.10 Vérifier le résultat
- 4.11 Copier: Copy-Item
- 4.12 Vérifier avec Get-ChildItem
- 4.13 Renommer: Rename-Item
- 4.14 Créer un sous-dossier
- 4.15 Déplacer: Move-Item
- 4.16 Vérifier le déplacement
- 4.17 Supprimer un fichier: Remove-Item
- 4.18 Tout voir d'un coup: le paramètre -Recurse
- 4.19 Supprimer un dossier vide
- 4.20 Supprimer un dossier et son contenu
- 4.21 Vérification finale

5. Récapitulatif des commandes

6. Difficultés rencontrées et solutions

7. Suite du cours

1. Contexte et objectifs

Ce volume rend compte de l'apprentissage de PowerShell, l'interpréteur de commandes de Windows, mené sous la forme d'un cours par modules. Chaque commande est présentée avec son rôle, sa syntaxe, son équivalent Linux lorsqu'il existe et une capture d'écran du résultat réel obtenu sur la machine de test.

L'objectif technique est de savoir administrer un système Windows en ligne de commande plutôt qu'avec l'interface graphique, ce qui est la base de l'automatisation et de l'administration de serveurs.

L'objectif pédagogique est de comprendre la logique du langage avant de mémoriser les commandes: un nom de commande suit toujours la forme *Verbe-Nom*, ce qui permet de deviner ce que fait une commande que l'on n'a jamais vue.

1.1 Qu'est-ce que PowerShell?

PowerShell est l'interpréteur de commandes de Microsoft: une fenêtre dans laquelle on tape des commandes pour piloter l'ordinateur, sans passer par la souris et les menus. Il est fourni avec Windows et repose sur la plateforme **.NET**. La version utilisée ici est **Windows PowerShell**, celle livrée avec Windows; une version plus récente, appelée simplement **PowerShell**, est open source et fonctionne aussi sous Linux et macOS.

Il remplace avantageusement l'ancienne invite de commandes (**cmd**), beaucoup plus limitée, et c'est aujourd'hui l'outil de référence pour administrer Windows et Windows Server.

1.2 À quoi sert PowerShell?

PowerShell sert à **administrer** un système et à **automatiser** des tâches répétitives. Ce que l'on fait en plusieurs clics dans l'interface graphique peut s'écrire en une ligne, puis se rejouer à l'identique sur un ou plusieurs ordinateurs.

- gérer les fichiers, les dossiers, les processus et les services de Windows
- configurer le réseau, les utilisateurs et les groupes locaux
- administrer Active Directory et les serveurs Windows, y compris à distance
- automatiser des tâches avec des scripts, par exemple la création de nombreux comptes utilisateurs d'un seul coup

1.3 Un shell et un langage de script

PowerShell est **les deux à la fois**. C'est d'abord un **shell** interactif: on tape une commande, on obtient le résultat immédiatement. C'est aussi un **langage de programmation de scripts**, avec des variables, des conditions, des boucles et des fonctions. Les commandes écrites dans un fichier **.ps1** forment un script que l'on peut exécuter à la demande (module 7 du cours).

Les commandes natives de PowerShell s'appellent des **cmdlets** (prononcer « commande-lettes »). Elles suivent toutes la forme Verbe-Nom, ce qui rend le langage très lisible et facile à explorer, comme on le verra au chapitre 3.

1.4 La particularité de PowerShell: les objets

Sous Linux ou dans l'ancienne invite de commandes, une commande renvoie du **texte**, que l'on doit découper pour en extraire une information. PowerShell renvoie des **objets**. Un objet est un élément structuré qui possède des **propriétés** (des informations sur lui) et des **méthodes** (des actions possibles sur lui).

Par exemple, **Get-ChildItem** renvoie un objet pour chaque fichier ou dossier. Les colonnes du tableau affiché (**Mode**, **LastWriteTime**, **Length**, **Name**) sont les propriétés de ces objets: la date de modification, la taille, le nom. Comme ce sont des propriétés et non du simple texte, on peut les filtrer, les trier ou les passer à une autre commande.

Cette idée est le cœur de PowerShell. Elle est exploitée avec le **pipeline** (|), qui transmet les objets d'une commande à la suivante: c'est le sujet du module 3.

Outil	Ce que renvoie une commande
Invite de commandes (cmd)	Du texte
Shell Linux (bash)	Du texte
PowerShell	Des objets, avec leurs propriétés

Périmètre couvert dans ce document

- Logique Verbe-Nom, recherche de commandes et aide intégrée
- Navigation dans le système de fichiers: emplacement, liste du contenu, changement de dossier
- Création de dossiers et de fichiers, écriture, ajout et lecture de contenu
- Copie, renommage, déplacement et suppression d'éléments, y compris récursive
- Vérification de chaque action par une commande de lecture

Les modules suivants (objets et pipeline, processus et services, scripts, réseau, Active Directory) sont présentés au chapitre 7.

2. Environnement technique

Toutes les commandes ont été exécutées sur un poste Windows, dans une console PowerShell ouverte en administrateur. Un dossier de test, créé au début du module 2 et supprimé à la fin, a servi de terrain d'essai afin de ne toucher à aucune donnée existante.

Élément	Détail
Système d'exploitation	Windows 11, interface en français
Console	Windows PowerShell, ouverte en administrateur
Accès Internet	Nécessaire uniquement pour télécharger l'aide (Update-Help)
Dossier de test	Test, créé puis supprimé dans le dossier personnel de l'utilisateur
Fichiers de test	notes.txt, copie.txt devenu vieux.txt, sous-dossier Archive

Méthode de travail

Chaque action est suivie d'une vérification: une commande de lecture (**Get-Content**, **Get-ChildItem**) confirme le résultat de chaque commande qui n'affiche rien elle-même. Les captures montrent uniquement la console, sans le reste de l'écran.

3. Les bases de PowerShell

Ce chapitre présente la logique de PowerShell et trois réflexes à acquérir: afficher une information, retrouver une commande dont on a oublié le nom et lire l'aide intégrée.

3.1 La logique Verbe-Nom: Get-Date

Chaque commande PowerShell, appelée **cmdlet**, suit la forme **Verbe-Nom**, avec un tiret. Le verbe dit ce que fait la commande, le nom dit sur quoi elle agit. **Get-Date** affiche la date et l'heure du système. Les verbes à retenir:

- **Get-**: obtenir, afficher
- **Set-**: modifier
- **New-**: créer
- **Remove-**: supprimer
- **Rename-**: renommer

Note. PowerShell ne distingue pas majuscules et minuscules. La documentation utilise pourtant toujours la forme officielle (**Get-Date**), comme dans la documentation Microsoft.



Figure 1. Get-Date: la date et l'heure du système

3.2 Retrouver une commande: Get-Command

Inutile de connaître toutes les commandes par cœur: **Get-Command** les cherche. Le caractère ***** remplace n'importe quelle suite de lettres, donc ***date*** trouve toutes les commandes dont le nom contient « date ».

Le résultat est long (ici, un extrait). La colonne **CommandType** indique le type (alias, fonction, cmdlet) et la colonne **Source** le module qui fournit la commande.

Syntaxe

```
Get-Command *<mot>*
```

```
PS C:\Users\Alex> Get-Command *date*

CommandType     Name                                               Version    Source
-----------     ----                                               -------    ------
Alias           Get-AppPackageAutoUpdateSettings                   2.0.1.0    Appx
Alias           Get-MsixPackageAutoUpdateSettings                  2.0.1.0    Appx
Alias           Remove-AppPackageAutoUpdateSettings                2.0.1.0    Appx
Alias           Remove-MsixPackageAutoUpdateSettings               2.0.1.0    Appx
Alias           Set-AppPackageAutoUpdateSettings                   2.0.1.0    Appx
Alias           Set-MsixPackageAutoUpdateSettings                  2.0.1.0    Appx
Function        Get-WindowsUpdateLog                               1.0.0.0    WindowsUpdate
Function        Update-AutologgerConfig                            1.0.0.0    EventTracingManagement
Function        Update-Disk                                        2.0.0.0    Storage
Function        Update-DscConfiguration                            1.1        PSDesiredStateConfiguration
Function        Update-EtwTraceSession                             1.0.0.0    EventTracingManagement
Function        Update-HostStorageCache                            2.0.0.0    Storage
Function        Update-IscsiTarget                                 1.0.0.0    iSCSI
Function        Update-IscsiTargetPortal                           1.0.0.0    iSCSI
Function        Update-Module                                      1.0.0.1    PowerShellGet
Function        Update-ModuleManifest                              1.0.0.1    PowerShellGet
Function        Update-MpSignature                                 1.0        ConfigDefender
Function        Update-MpSignature                                 1.0        Defender
Function        Update-NetFirewallDynamicKeywordAddress            2.0.0.0    NetSecurity
Function        Update-NetIPsecRule                                2.0.0.0    NetSecurity
Function        Update-Script                                      1.0.0.1    PowerShellGet
Function        Update-ScriptFileInfo                              1.0.0.1    PowerShellGet
Function        Update-SmbMultichannelConnection                   2.0.0.0    SmbShare
Function        Update-StorageBusCache                             1.0.0.0    StorageBusCache
Function        Update-StorageFirmware                             2.0.0.0    Storage
Function        Update-StoragePool                                 2.0.0.0    Storage
```

*Figure 2. Get-Command *date*: toutes les commandes contenant « date » (extrait)*

3.3 L'aide intégrée: Get-Help

Get-Help suivi du nom d'une commande affiche son manuel (nom, résumé, syntaxe, description). Avec le paramètre **-Examples**, seuls les exemples d'utilisation sont affichés, ce qui est souvent le plus utile pour apprendre. L'aide s'affiche en anglais, car les fichiers d'aide téléchargés sont en anglais: c'est normal.

Syntaxe

```
Get-Help <Commande> -Examples
```

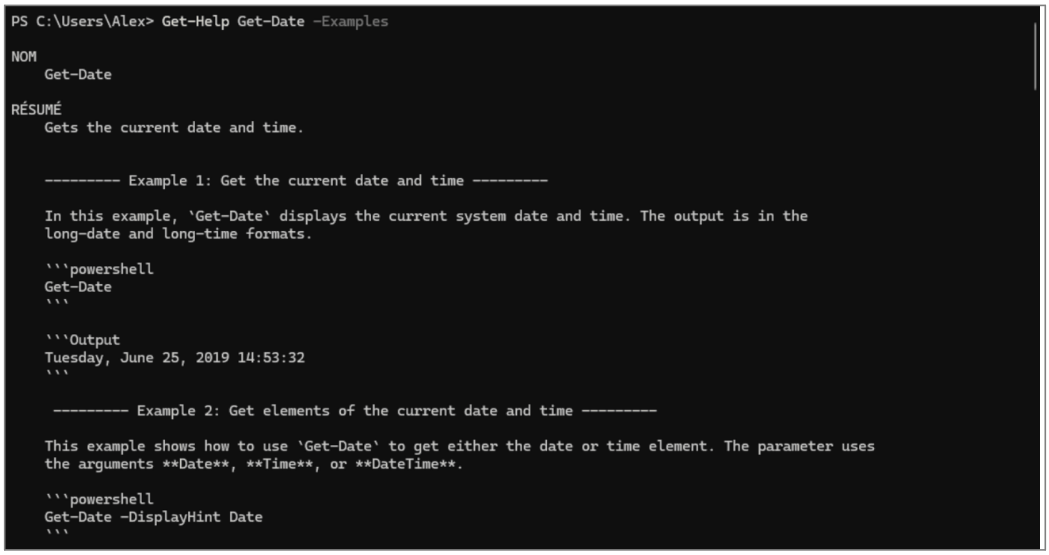


Figure 3. Get-Help Get-Date -Examples: les exemples d'utilisation (extrait)

3.4 Télécharger l'aide: Update-Help

Les fichiers d'aide ne sont pas fournis en entier avec Windows. **Update-Help** les télécharge: elle se lance dans une fenêtre PowerShell ouverte **en administrateur**, avec un accès Internet. Une barre de progression indique le module en cours de mise à jour.

Message rouge. La mise à jour se termine par un message d'erreur en rouge à propos du module **Microsoft.PowerShell.Host** (culture en-US). C'est un problème connu et sans conséquence: l'aide des autres commandes est bien installée. Voir le chapitre 6.



Figure 4. Update-Help: téléchargement des fichiers d'aide (barre de progression)

3.5 Effacer l'écran: cls

cls (alias de **Clear-Host**, équivalent de **clear** sous Linux) efface l'écran de la console. Pratique pour repartir d'une fenêtre propre avant de lancer une commande ou de prendre une capture.



Figure 5. cls: la commande est tapée, l'écran sera effacé

4. Fichiers et dossiers

Ce chapitre couvre la gestion du système de fichiers: se repérer, se déplacer, créer, écrire, lire, copier, renommer, déplacer et supprimer. Un dossier de test **Test** sert de fil conducteur, et il est supprimé à la fin.

4.1 Où suis-je? Get-Location

PowerShell travaille toujours « dans » un dossier. **Get-Location** affiche le dossier courant, l'équivalent de **pwd** sous Linux.

```
PS C:\Users\Alex> Get-Location

Path
----
C:\Users\Alex


PS C:\Users\Alex>
```

Figure 6. Get-Location: le dossier courant

4.2 Lister le contenu: Get-ChildItem

Get-ChildItem liste les fichiers et dossiers de l'endroit où l'on se trouve, l'équivalent de **ls**.

La colonne **Mode** se lit lettre par lettre: un **d** au début signale un dossier (directory), un **-** un fichier. Un **a** signale l'attribut Archive, que Windows pose sur les fichiers ordinaires, et un **r** la lecture seule.

```
PS C:\Users\Alex> Get-ChildItem


    Répertoire : C:\Users\Alex


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-r---          22-04-26     19:03                Contacts
d-r---          22-04-26     19:03                Desktop
d-r---          09-10-26     10:45                Documents
d-r---          22-04-26     19:03                Downloads
d-r---          22-04-26     19:03                Favorites
d-r---          22-04-26     19:03                Links
d-r---          22-04-26     19:03                Music
d-r---          22-04-26     19:06                OneDrive
d-r---          22-04-26     19:03                Pictures
d-r---          22-04-26     19:03                Saved Games
d-r---          22-04-26     19:53                Searches
d-r---          08-10-26     23:48                Videos
```

Figure 7. Get-ChildItem: le contenu du dossier personnel de l'utilisateur

4.3 Changer de dossier: Set-Location

Set-Location déplace dans un dossier, l'équivalent de **cd**. Après la commande, l'invite affiche le nouveau chemin. **cd** fonctionne aussi: c'est un **alias**, un raccourci vers Set-Location, pratique pour taper vite en direct. Dans la documentation et les scripts, on écrit le nom complet.

Syntaxe

```
Set-Location <chemin>
```

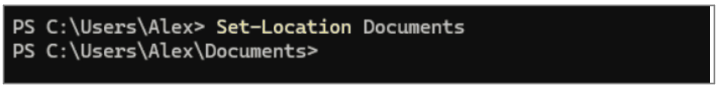


Figure 8. Set-Location Documents: on descend dans Documents

4.4 Remonter d'un niveau: Set-Location ..

Les deux points **..** désignent le dossier parent. L'invite repasse de **Documents** au dossier personnel.



Figure 9. Set-Location ..: on remonte au dossier parent

4.5 Créer un dossier: New-Item

New-Item crée un élément. Le paramètre **-ItemType** précise sa nature (**Directory** pour un dossier, **File** pour un fichier) et **-Name** son nom. Le dossier est créé à l'endroit où l'on se trouve. Le tableau affiché confirme la création, avec un **d** dans la colonne Mode.

Syntaxe

```
New-Item -ItemType Directory -Name <nom>
```

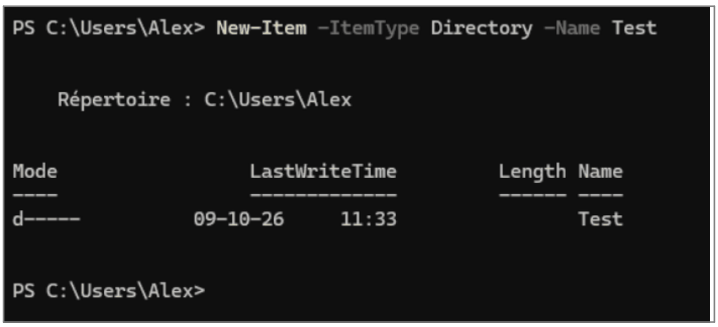


Figure 10. New-Item -ItemType Directory: création du dossier Test

4.6 Créer un fichier: New-Item -ItemType File

Pour un fichier, on remplace **Directory** par **File** et on ajoute **-Path** (le chemin) pour indiquer le dossier de destination. Le fichier est vide: sa taille (Length) est de 0.

Ordre des paramètres. Pour les paramètres nommés (**-ItemType**, **-Name**, **-Path**), l'ordre n'a pas d'importance. Il compte seulement quand on n'écrit pas le nom du paramètre.

Syntaxe

```
New-Item -ItemType File -Name <nom> -Path <dossier>
```

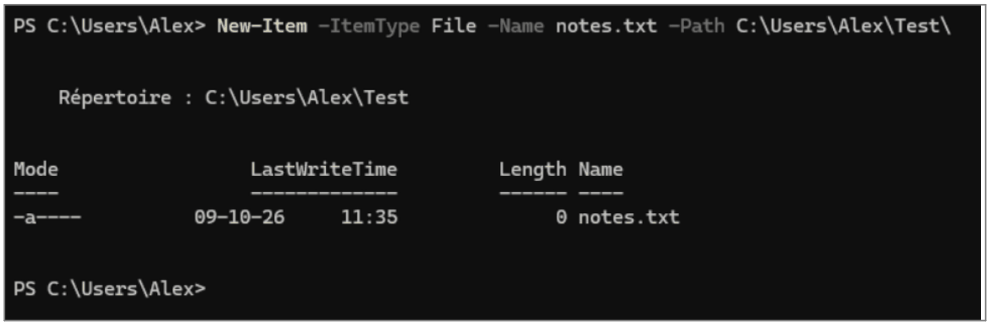


Figure 11. New-Item -ItemType File: création de notes.txt dans Test

4.7 Écrire dans un fichier: Set-Content

Set-Content écrit le texte donné par **-Value** dans le fichier désigné par **-Path**. Attention: elle **remplace** tout le contenu existant. Elle n'affiche rien, ce qui est normal.

Syntaxe

```
Set-Content -Path <fichier> -Value "<texte>"
```



Figure 12. Set-Content: écriture de « Bonjour PowerShell » dans notes.txt

4.8 Lire un fichier: Get-Content

Get-Content affiche le contenu d'un fichier, l'équivalent de **cat**. Le nom du fichier suffit, **-Path** est facultatif.



Figure 13. Get-Content: lecture de notes.txt

4.9 Ajouter une ligne: Add-Content

Add-Content ajoute du texte à la **fin** du fichier sans effacer l'existant, à la différence de Set-Content.

Syntaxe

```
Add-Content -Path <fichier> -Value "<texte>"
```



Figure 14. Add-Content: ajout de « Deuxième ligne »

4.10 Vérifier le résultat

Une nouvelle lecture avec **Get-Content** montre les deux lignes. Ici, le paramètre **-Path** est écrit explicitement: le résultat est identique.

```
PS C:\Users\Alex> Get-Content -Path C:\Users\Alex\Test\notes.txt
Bonjour PowerShell
Deuxième ligne
PS C:\Users\Alex>
```

Figure 15. Get-Content -Path: le fichier contient maintenant deux lignes

4.11 Copier: Copy-Item

Copy-Item prend une **source** puis une **destination**. Ici, la source et la copie sont dans le même dossier. Sans noms de paramètres, l'ordre est important. Il faut donner le chemin des deux éléments.

Syntaxe

```
Copy-Item <source> <destination>
```

```
PS C:\Users\Alex> Copy-Item C:\Users\Alex\Test\notes.txt C:\Users\Alex\Test\copie.txt
PS C:\Users\Alex>
```

Figure 16. Copy-Item: copie de notes.txt vers copie.txt

4.12 Vérifier avec Get-ChildItem

Une fois le chemin du dossier donné à **Get-ChildItem**, la liste montre les deux fichiers, de taille identique (36 octets).

```
PS C:\Users\Alex> Get-ChildItem C:\Users\Alex\Test\


    Répertoire : C:\Users\Alex\Test


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----          09-10-26     11:39             36 copie.txt
-a----          09-10-26     11:39             36 notes.txt


PS C:\Users\Alex>
```

Figure 17. Get-ChildItem: notes.txt et copie.txt dans Test

4.13 Renommer: Rename-Item

Rename-Item prend l'ancien chemin puis le nouveau nom, sans chemin pour le nouveau nom. La commande n'affiche rien; le résultat est visible à la figure 21, où le fichier porte son nouveau nom.

Syntaxe

```
Rename-Item <ancien chemin> <nouveau nom>
```

```
PS C:\Users\Alex> Rename-Item C:\Users\Alex\Test\copie.txt vieux.txt
PS C:\Users\Alex>
```

Figure 18. Rename-Item: copie.txt devient vieux.txt

4.14 Créer un sous-dossier

Même commande qu'à la section 4.5, cette fois avec **-Path** pour créer le dossier **Archive** à l'intérieur de **Test**.

```
PS C:\Users\Alex> New-Item -ItemType Directory -Path C:\Users\Alex\Test\ -Name Archive


    Répertoire : C:\Users\Alex\Test


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----          09-10-26     11:44                Archive


PS C:\Users\Alex>
```

Figure 19. New-Item: création du sous-dossier Archive

4.15 Déplacer: Move-Item

Move-Item déplace un élément vers une destination, l'équivalent de **mv**. Même ordre qu'avec Copy-Item: source, puis destination.

Syntaxe

```
Move-Item <source> <destination>
```

```
PS C:\Users\Alex> Move-Item C:\Users\Alex\Test\vieux.txt C:\Users\Alex\Test\Archive\
PS C:\Users\Alex>
```

Figure 20. Move-Item: vieux.txt déplacé dans Archive

4.16 Vérifier le déplacement

Le contenu du dossier **Archive** montre que **vieux.txt** s'y trouve bien.

```
PS C:\Users\Alex> Get-ChildItem C:\Users\Alex\Test\Archive\


    Répertoire : C:\Users\Alex\Test\Archive


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----          09-10-26     11:39             36 vieux.txt


PS C:\Users\Alex>
```

Figure 21. Get-ChildItem: vieux.txt dans Archive

4.17 Supprimer un fichier: Remove-Item

Remove-Item supprime exactement l'élément désigné par le chemin, l'équivalent de **rm**. Ici, seul le fichier **vieux.txt** est visé. **Il n'y a pas de corbeille: la suppression est définitive.**

Syntaxe

```
Remove-Item <chemin>
```

```
PS C:\Users\Alex> Remove-Item C:\Users\Alex\Test\Archive\vieux.txt
PS C:\Users\Alex>
```

Figure 22. Remove-Item: suppression de vieux.txt

4.18 Tout voir d'un coup: le paramètre -Recurse

Avec **-Recurse** (récursif), Get-ChildItem liste le dossier **et** le contenu de ses sous-dossiers. L'en-tête **Répertoire:** indique le dossier dont on voit le contenu. Ici, le dossier **Test** contient **Archive** et **notes.txt**. Aucun bloc supplémentaire n'est affiché pour **Archive**, ce qui montre qu'il est vide: un dossier vide n'affiche rien.

Syntaxe

```
Get-ChildItem <chemin> -Recurse
```

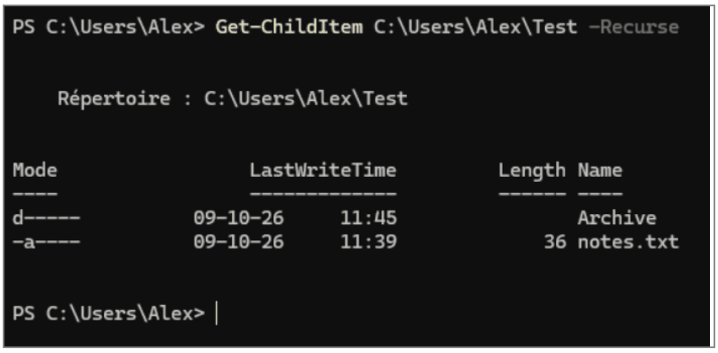


Figure 23. Get-ChildItem -Recurse: Test contient Archive (vide) et notes.txt

4.19 Supprimer un dossier vide

Un dossier vide se supprime sans confirmation.



Figure 24. Remove-Item: suppression du dossier Archive

4.20 Supprimer un dossier et son contenu

Pour supprimer un dossier qui contient des éléments, on ajoute **-Recurse**. Sans ce paramètre, PowerShell demande une confirmation. **Cette action est définitive.**

Syntaxe

```
Remove-Item <chemin> -Recurse
```

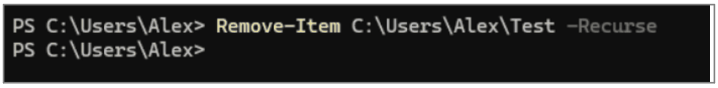


Figure 25. Remove-Item -Recurse: suppression du dossier Test et de son contenu

4.21 Vérification finale

Le contenu du dossier personnel ne contient plus **Test**: l'environnement de test est propre.

```
PS C:\Users\Alex> Get-ChildItem C:\Users\Alex\


    Répertoire : C:\Users\Alex


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-r---        22-04-26     19:03                Contacts
d-r---        22-04-26     19:03                Desktop
d-r---        09-10-26     10:45                Documents
d-r---        22-04-26     19:03                Downloads
d-r---        22-04-26     19:03                Favorites
d-r---        22-04-26     19:03                Links
d-r---        22-04-26     19:03                Music
d-r---        22-04-26     19:06                OneDrive
d-r---        22-04-26     19:03                Pictures
d-r---        22-04-26     19:03                Saved Games
d-r---        22-04-26     19:53                Searches
d-r---        08-10-26     23:48                Videos


PS C:\Users\Alex>
```

Figure 26. Get-ChildItem: le dossier Test a disparu

5. Récapitulatif des commandes

Tableau de référence des commandes vues dans les modules 1 et 2, avec leur équivalent Linux.

Commande	Rôle	Équivalent Linux
`Get-Date`	Affiche la date et l'heure	date
`Get-Command *mot*`	Cherche une commande par un mot de son nom	apropos
`Get-Help Nom -Examples`	Affiche l'aide (exemples seuls avec -Examples)	man
`Update-Help`	Télécharge les fichiers d'aide (administrateur)	aucun
`cls`	Efface l'écran (alias de Clear-Host)	clear
`Get-Location`	Affiche le dossier courant	pwd
`Get-ChildItem`	Liste le contenu d'un dossier (-Recurse: sous-dossiers inclus)	ls (ls -R)
`Set-Location`	Change de dossier	cd
`New-Item`	Crée un fichier ou un dossier (-ItemType)	touch, mkdir
`Set-Content`	Écrit dans un fichier, en remplaçant le contenu	echo >
`Add-Content`	Ajoute du texte à la fin d'un fichier	echo >>
`Get-Content`	Affiche le contenu d'un fichier	cat
`Copy-Item`	Copie (source puis destination)	cp

Commande	Rôle	Équivalent Linux
`Rename-Item`	Renomme	mv (renommage)
`Move-Item`	Déplace (source puis destination)	mv
`Remove-Item`	Supprime, sans corbeille (-Recurse pour un dossier plein)	rm (rm -r)

Points de vigilance

- **Pas de corbeille**: Remove-Item supprime définitivement.
- **-Recurse** (et non « recursive »): nécessaire pour supprimer un dossier plein ou pour lister les sous-dossiers.
- **Set-Content remplace, Add-Content ajoute**: ne pas les confondre.
- **Chemins complets**: avec Copy-Item, Move-Item et Rename-Item, donner le chemin de chaque fichier si l'on n'est pas dans son dossier.
- **Casse et alias**: PowerShell ignore la casse et accepte des alias comme cd, ls ou cat; la documentation utilise les noms complets.

6. Difficultés rencontrées et solutions

Deux points ont demandé une manipulation particulière lors du module 1. Ils sont documentés ici pour pouvoir les reproduire et les résoudre rapidement.

Anomalie	Cause	Solution
Get-Help n'affiche presque rien (ou une aide très partielle)	Les fichiers d'aide ne sont pas installés sur la machine.	Lancer **Update-Help** dans une console ouverte en administrateur, avec accès Internet, puis relancer Get-Help.
Update-Help se termine par un message rouge sur le module Microsoft.PowerShell.Host (culture en-US)	Le téléchargement de l'aide de ce module échoue pour la culture en-US.	Sans conséquence: l'aide des autres commandes est installée. Vérifier avec **Get-Help Get-Date -Examples**.

7. Suite du cours

Le cours compte 12 modules, chacun terminé par une série d'exercices. Ce volume 1 couvre les modules 1 et 2. La suite est répartie en deux volumes, publiés au fil de l'avancement.

Module	Thème	Volume
1	Les bases de PowerShell	1
2	Fichiers et dossiers	1
3	Les objets et le pipeline (|)	2
4	Filtrer, trier, sélectionner	2
5	Processus et services	2
6	Variables, boucles, conditions	2
7	Scripts (.ps1) et fonctions	3
8	Réseau	3
9	Utilisateurs et groupes locaux	3
10	Active Directory	3
11	Administration à distance	3
12	Automatisation et gestion des erreurs	3